

DevOps Lab

Documentation

Program 9:

Organize Kubernetes Pods using Labels and Namespace

The program aims to logically organize, manage, and control Kubernetes Pods by using Namespaces and Labels, making applications easier to deploy, monitor, and maintain in a cluster.

Project Structure:

```
program_9/  
—>namespaces.yaml  
—>pods.yaml
```

Procedure:

Create a folder (ex: program_9)

```
1RV24MC062_lubna_tabassum@Inspiron:~$ cd DevOps  
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps$ mkdir program_9  
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps$ cd program_9  
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ nano namespaces.yaml  
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ nano pods.yaml
```

Step 1: Start Minikube

```
minikube start
```

Check the status of Minikube

```
minikube status
```

```

1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ minikube start
🐳 minikube v1.37.0 on Ubuntu 24.04
🌟 Using the docker driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
🚢 Pulling base image v0.0.48 ...
🏃 Updating the running docker "minikube" container ...
🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔍 Verifying Kubernetes components...
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌞 Enabled addons: storage-provisioner, default-storageclass

! /usr/local/bin/kubectl is version 1.31.0, which may have incompatibilities with Kubernetes 1.34.0.
   ▪ Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
🏁 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

```

Step 2: Create namespaces.yaml file with the following code

nano namespaces.yaml

```

GNU nano 7.2 namespaces.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: prod

```

[Read 10 lines]

[^]G Help [^]O Write Out [^]W Where Is [^]K Cut [^]T Execute [^]C Location
[^]X Exit [^]R Read File [^]_ Replace [^]U Paste [^]J Justify [^]_ Go To Line

Step 3: Create pods.yaml file with the following code

nano pods.yaml

```
Dec 17 14:59
1RV24MC062_lubna_tabassum@inspiron: ~/DevOps/program_9
GNU nano 7.2 pods.yaml
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: dev
  labels:
    app: frontend
    env: dev
spec:
  containers:
    - name: frontend
      image: nginx
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: dev
  labels:
    app: backend
[ Read 59 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

```
Dec 17 14:59
1RV24MC062_lubna_tabassum@inspiron: ~/DevOps/program_9
GNU nano 7.2 pods.yaml
  env: dev
spec:
  containers:
    - name: backend
      image: nginx
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: prod
  labels:
    app: frontend
    env: prod
spec:
  containers:
    - name: frontend
      image: nginx
      ports:
        - containerPort: 80
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Step 4: Apply the files

```
kubectl apply -f namespaces.yaml
```

```
kubectl apply -f pods.yaml
```

```
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl apply -f namespaces.yaml
namespace/dev unchanged
namespace/prod unchanged
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl apply -f pods.yaml
pod/frontend-pod created
pod/backend-pod created
pod/frontend-pod created
pod/backend-pod created
```

Step 5: Verify the pods with namespaces

```
kubectl get pods -n dev
```

```
kubectl get pods -n prod
```

```
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl get pods -n dev
NAME          READY   STATUS    RESTARTS   AGE
backend-pod   1/1     Running   0           2m18s
frontend-pod  1/1     Running   0           2m18s
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl get pods -n prod
NAME          READY   STATUS    RESTARTS   AGE
backend-pod   1/1     Running   0           2m23s
frontend-pod  1/1     Running   0           2m23s
```

Step 6: Get all of the pods

```
docker get pods -A
```

```
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl get pods -A
NAMESPACE     NAME          READY   STATUS    RESTARTS   AGE
dev            backend-pod   1/1     Running   0           2m26s
dev            frontend-pod  1/1     Running   0           2m26s
kube-system    coredns-66bc5c9577-6zbs6  1/1     Running   5 (5m6s ago)  2d5h
kube-system    etcd-minikube  1/1     Running   5 (5m4s ago)  2d5h
kube-system    kube-apiserver-minikube  1/1     Running   5 (5m1s ago)  2d5h
kube-system    kube-controller-manager-minikube  1/1     Running   5 (5m11s ago)  2d5h
kube-system    kube-proxy-xbjt  1/1     Running   5 (5m11s ago)  2d5h
kube-system    kube-scheduler-minikube  1/1     Running   5 (5m11s ago)  2d5h
kube-system    storage-provisioner  1/1     Running   10 (4m58s ago)  2d5h
prod           backend-pod   1/1     Running   0           2m26s
prod           frontend-pod  1/1     Running   0           2m26s
```

Step 7: Verify pods with labels

```
kubectl get pods -n dev -l app=frontend
```

```
kubectl get pods -n dev -l app=backend
```

```
kubectl get pods -n prod -l app=frontend
```

```
kubectl get pods -n prod -l app=backend
```

```
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl get pods -n dev -l app=frontend
NAME          READY   STATUS    RESTARTS   AGE
frontend-pod  1/1     Running   0           2m45s
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl get pods -n dev -l app=backend
NAME          READY   STATUS    RESTARTS   AGE
backend-pod    1/1     Running   0           2m50s
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl get pods -n prod -l app=frontend
NAME          READY   STATUS    RESTARTS   AGE
frontend-pod  1/1     Running   0           3m
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ kubectl get pods -n prod -l app=backend
NAME          READY   STATUS    RESTARTS   AGE
backend-pod    1/1     Running   0           3m6s
1RV24MC062_lubna_tabassum@Inspiron:~/DevOps/program_9$ █
```